

Project Orihime

Project Orihime is an agentic post-production engine designed for automated quality control (QC) and asset reconstruction. It leverages Gemini 1.5 Pro as an autonomous orchestrator that interprets telemetry from ClickHouse and Grafana to perform surgical video corrections, 12-bit HDR expansion, and frame-level healing.

1. Agentic Orchestration & MCP Integration

The core of Orihime is not a script, but a reasoning agent. Gemini 1.5 Pro utilizes the Model Context Protocol (MCP) to interact with the environment as a decision-maker rather than a static processor.

- **ClickHouse MCP Tool:** Gemini queries ClickHouse to retrieve frame-by-frame metadata, including bit-depth analysis, peak brightness levels, and missing frame sequences. It does not just display this data; it uses the results to calculate required color offsets.
- **Grafana MCP Tool:** Gemini monitors real-time rendering telemetry. If performance dips below 24fps during a reconstruction pass, the agent autonomously adjusts the FFmpeg threading model or switches filter kernels to maintain system stability.
- **Dynamic Command Generation:** The agent is responsible for writing complex FFmpeg `filter_complex` strings. Based on the QC analysis, it calculates 3D LUT coefficients and dynamically generates the zscale filter parameters for HDR10/HLG mapping.

2. High-Performance 4K Processing Strategy

To solve the 4K bottleneck, Project Orihime bypasses the overhead of Python-based pixel manipulation. All mathematical heavy lifting is offloaded to native C implementations.

- **Native C-Filters:** We utilize FFmpeg’s `zscale` filter, which implements the Zimg library for high-performance, high-bit-depth image scaling, colorspace conversion, and bit-depth expansion (8-bit to 12-bit).
- **Elimination of NumPy/OpenCV:** Pixel-level math is handled via compiled FFmpeg bitstream filters and custom 3D LUTs. This ensures 4K 60fps throughput that would be impossible with interpreted Python loops or CPU-bound NumPy arrays.
- **Hardware Acceleration:** The backend targets NVENC/NVDEC with `cuda` filters for color conversion, ensuring the data stays on the GPU memory bus throughout the reconstruction pipeline.

3. Reconstruction & Temporal Integrity

The system differentiates between isolated "dead" frames and continuous temporal degradation to prevent the "boiling" effect common in AI video tools.

- **Isolated Healing (Imagen 3):** When the agent identifies a single corrupted or missing frame (a "hole" in the timeline), it invokes Imagen 3 with a frame-accurate reference prompt to regenerate the specific asset.
- **Temporal Stability (Motion Interpolation):** For sequences involving motion or continuous frame issues, Orihime avoids per-frame AI generation. Instead, it uses FFmpeg’s `minterpolate` filter with motion-compensated pixel prediction. This maintains the vector consistency of the original footage and eliminates flickering.

4. Technical Architecture Overview

Component	Responsibility	Technology Stack
Brain	Intent reasoning & tool use	Gemini 1.5 Pro
Telemetry	Metadata & QC data storage	ClickHouse (via MCP)

Component	Responsibility	Technology Stack
Observability	Runtime performance tracking	Grafana (via MCP)
Execution Engine	High-speed video processing	FFmpeg (C-filters, zscale)
Frame Healer	Single-frame reconstruction	Imagen 3
Storage	Raw 4K Video Assets	Local NVMe / Fast Scratch Disk

5. 72-Hour Sprint Roadmap

This roadmap focuses on a functional prototype built for local deployment and immediate demonstration.

Day 1: Infrastructure & Data Ingest

- **Hour 0-6:** Set up the local environment with FFmpeg (compiled with zscale and CUDA support).
- **Hour 6-12:** Initialize ClickHouse and define the schema for video metadata (Luma, Chroma, Frame ID).
- **Hour 12-24:** Build the MCP connectors for ClickHouse and Grafana to expose data to Gemini 1.5 Pro.

Day 2: Agentic Logic & Command Synthesis

- **Hour 24-30:** Prompt engineering for Gemini 1.5 Pro to interpret QC failures from ClickHouse logs.
- **Hour 30-42:** Implement the "Command Factory"—the logic where Gemini writes and executes FFmpeg LUT/filter commands.
- **Hour 42-48:** Integrate Imagen 3 API for the isolated frame reconstruction module.

Day 3: Integration, QC, & Demo

- **Hour 48-60:** Fine-tune motion interpolation parameters in FFmpeg to ensure seamless transitions between original and healed frames.
- **Hour 60-66:** Stress-test the 4K pipeline; optimize zscale kernels for HDR expansion performance.
- **Hour 66-72:** Finalize the demo UI (Streamlit or similar) to visualize the Agent's thought process and the resulting reconstructed 4K footage.

6. Implementation Notes

The system must ensure that the `zscale` filter is configured for `dither=error_diffusion` when upscaling from 8-bit to 12-bit to prevent banding in the final HDR output. All FFmpeg commands generated by Gemini should prioritize `-pix_fmt yuv420p10le` or higher to maintain the integrity of the reconstructed color volume.